AMRITA VISHWA VIDYAPEETHAM

Course Code: 23ECE337

Course Name: System On Chip

Component: Simulation and Analysis-based Assignment

Date of Evaluation:

Academic Year: 2026-2027 (Odd Semester)

Batch Number:

Name of the Students: 1) C. Deepthi Sree (BL.EN.U4ECE23107)

With Registration no. 2) G. Sohan (BL.EN.U4ECE23117)

3) K. Lochana Reddy (BL.EN.U4ECE23127)

Branch: ECE and EAC

Semester: VII

Batch: 2023-2027

Faculty In-charge:

Aim:

To model and formally verify the one-master one-slave (1M1S) configuration of a SystemC/TLM 2.0 model of the ARM AMBA AHB bus using UPPAAL timed automata, and to compare the results with the 1M1S column of Table II in the IEEE paper by Pockrandt, Herber and Glesner (ESTIMedia 2011) [1]. The three properties of that table that apply to 1M1S are checked: **deadlock freedom**, **bus grant to master 1** and **communication in time**. In addition, the missing `m_txn_count` reset reported in the paper is recreated and analysed.

Tools Used:

- **UPPAAL 5.0.0** (academic licence): the `verifyta` command-line model checker produced all reported numbers; the UPPAAL GUI (Editor, Verifier and Symbolic Simulator) was used for the screenshots.
- **UPPAAL timed-automata model of the Carbon Design Systems SystemC/TLM 2.0 AMBA AHB design**, produced by the SystemC-to-UPPAAL transformation of [1], [2] and reduced to 1M1S.
- **Python 3.12** scripts written for this assignment: `make_models.py` (adds the timing clock and builds the bug-study model), `run_all.py` (runs every check, saves the raw output, writes the results), `make_figures.py` and `make_clean_view.py` (figures and GUI-friendly demo copies).
- **Java 21 (Eclipse Temurin)** for the UPPAAL 5 graphical interface.
- **Host:** Windows 11, Intel64 Family 6 Model 154 Stepping 3, GenuineIntel.

Theory:

1) Introduction

Transaction-level models (TLM) in SystemC let a system on chip be simulated long before RTL exists. A simulation, however, follows only one ordering of the concurrent processes, so corner cases such as a counter that is not reset after a transaction can go unnoticed. Model checking explores every reachable state of a formal model instead, and either proves a property or returns a counterexample.

Reference [1] transforms SystemC/TLM 2.0 designs into UPPAAL timed automata and applies the method to an industrial TLM model of the AMBA AHB. Its key contribution is an optimised encoding of the TLM payload event queue (PEQ): each pending notification is held by its own automaton with a local clock instead of a global clock array, which reduces the state space. The paper checks the configurations 1M1S to 2M2S. For 1M1S it reports 74,345 states, 0:07 for deadlock freedom, 0:05 for bus grant to M1 and 0:17 for communication in time. This assignment repeats the 1M1S experiment.

2) Block Diagram

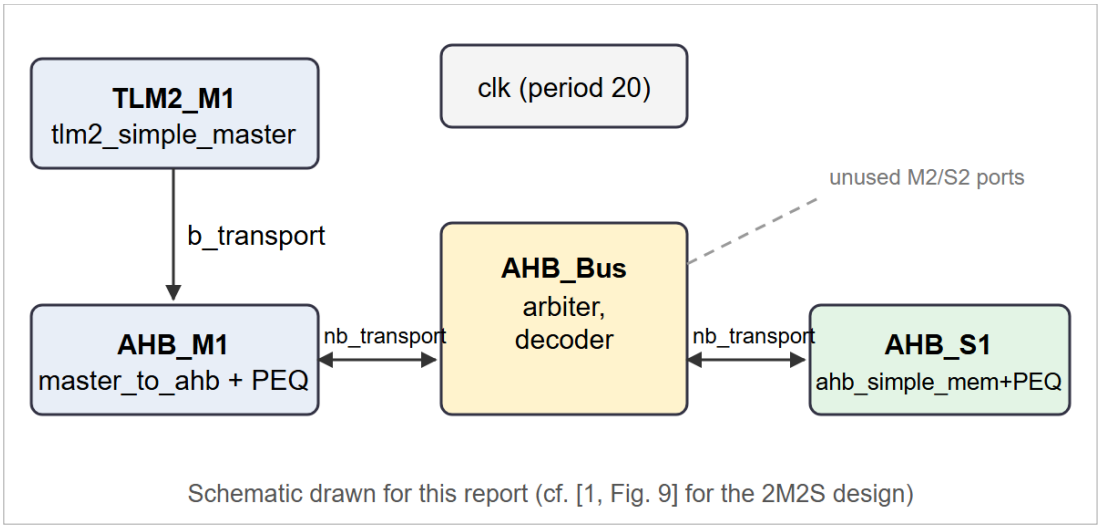


Fig. 1. 1M1S configuration verified in this work (schematic; cf. [1, Fig. 9]).

Block	UPPAAL instance	Function
Traffic generator	TLM2_M1 (tlm2_simple_master)	Writes and then reads a 3-beat burst at address 0, waits (wait_clks = 20) and repeats, using blocking transport
Master adapter	AHB_M1 (tlm2_master_to_ahb) + PEQ	Splits each transaction into AHB transfers; requests the bus and sends/receives data with nb_transport
Bus	AHB_Bus (ahb_simple_bus)	Arbiter (grant register m_InService_Register) and address decoder
Slave	AHB_S1 (ahb_simple_mem) + PEQ	Memory (base 0, size 3) that serves reads and writes
Clock and kernel	clk, scheduler, initialiser	Bus clock with period 20 time units; SystemC scheduler semantics (evaluate, update, advance time)

In total the model has 66 automata built from 44 templates. It was reduced by hand from the 2M2S model: the master-2, slave-2 and second traffic-generator instances were removed, `number_of_slaves` was set to 1 and the initial `readyprocs` to 2. The bus templates still loop over two master slots and two slave slots; the unused ports are not connected to any automaton.

3) Working Principle

SystemC to timed automata. Each SystemC method becomes a timed-automaton template; processes, events and the scheduler are predefined automata that synchronise over UPPAAL channels (for example `activate`, `deactivate`, `advancetime`, `deltadelay`). Urgent and committed locations model zero-time delta cycles.

PEQ encoding. In our model each PEQ consists of `peq_notify` (accepts a notification and hands it to a free element), four `peq_element` automata (each holds one notification with local clock `c` and the invariant `c <= delay`, corresponding to [1, Fig. 4]) and `peq_fetch` (calls the callback, [1, Fig. 5]).

Properties (TCTL). `A[] p` means p holds in every reachable state; `E<> p` means some run reaches p; `p --> q` (leads-to) means whenever p holds, q eventually holds. Deadlock freedom is `A[] not deadlock`. Bus grant is the leads-to property `AHB_M1$m_bus_req_pending --> AHB_M1$m_bus_granted`.

Communication in time. As in [1], an extra clock `x` is added and reset whenever the master is granted the bus; a flag `M1_isCommunicating` is true from a grant until the transaction is cleaned

up. The property `A[] M1_isCommunicating imply x <= N` is checked, and the smallest N that holds is found by doubling from one clock period and then bisecting. The two new variables are only written, never read, so the behaviour of the model is unchanged.

Bug study. The paper reports that a counter storing the number of completed transfers was not reset, so all split transactions after the first one failed. In our model this counter is `m_txn_count`, reset at `tlm2_master_to_ahb.cpp` lines 120 and 154 (the UPPAAL model merges each clean-up into one update, annotated with lines 116 and 150). A second copy of the timed model has both resets removed.

4) Advantages

- Exhaustive: every reachable state is explored (1,241,387 states for deadlock freedom), not just one simulation run.
- A failing property comes with a counterexample trace that can be stepped through in the simulator.
- Timing properties can be proved exactly: the communication-in-time bound N is shown to be the smallest value that holds.
- The transformation keeps the structure of the SystemC design, so results map back to source lines.
- Fully reproducible: one script reruns every check and regenerates the results from the raw output.

5) Disadvantages

- State-space explosion: the paper reports 74,345 states for 1M1S but 7,964,195 for 2M2S.
- Only bounded integers and static processes are supported, so the original design had to be simplified (no dynamic memory, simplified transaction objects).
- Results depend on the model: our 1M1S model was reduced by hand, and its state count differs from the paper's.
- Deadlock freedom alone is not enough: in the bug study the missing reset causes no deadlock and is only caught by a liveness property.
- Tool issues: UPPAAL 4.1.26-1 failed its start-up check, and the UPPAAL 5 GUI cannot handle "$" in the generated names, so GUI demo copies with "__" were needed.

SystemC Code:

The SystemC/TLM 2.0 design is the AMBA AHB example of the STATE-2.1 tool (`examples/amba_AHB`). Its sources (bus, memory slave, master adapter, traffic generator and `defines.h`) are included in `systemc_source/`; excerpts (a) to (d) below are copied from them with their real line numbers. Excerpts (e) and (f) are the code we added.

a) Bus arbiter: ahb_simple_bus::arbitrate (ahb_simple_bus.cpp)

The arbiter grants the bus to the requesting master with the lowest index, and only when the bus is free (`m_InService_Register == 0`, line 14). The loop runs over `MASTER_N` master slots (line 9), and `defines.h` sets `MASTER_N` to 2 (line 8). Our 1M1S model kept the bus unchanged, so it still has two slots, of which only slot 0 is ever requested.

```
  3  void ahb_simple_bus::arbitrate()
  4  {// Currently Grant is being given to the master with lower index
  5  // we are not preempting the onging txn in this arbiter even if higher priority master
sends req
  6    DEBUG_OUT("arbitrate BEGIN");
  7    //TODO mp: i had to change this as UPPAAL does not support numbers > 2^16-1 by
default. This is now depending on the number of masters which should work fine.
  8    //for(unsigned short j = 0; j < 16; j++)
  9    for(unsigned short j = 0; j < MASTER_N; j++)
 10    {
 11      unsigned short temp = 1 << j;
 12      if(m_BusRequest_Register & temp)
 13      {
 14        if(m_InService_Register == 0)
 15        {
 16          m_InService_Register= temp;
 17          current_req_master=j;
 18          //clear the jth bit in m_BusRequest_Register now
 19          //m_BusRequest_Register &= (~temp); clear is done via BUS_REQ_STOP
 20          m_BusRequest_Register -= temp;
 21        }
 22      }
 23    }
 24    DEBUG_OUT("arbitrate END");
 25  }
```

b) Master adapter: tlm2_master_to_ahb::b_transport, transaction clean-up (tlm2_master_to_ahb.cpp)

After a write (lines 109-124) and after a read (lines 143-158) the adapter releases the transaction and resets `m_txn_count` (lines **120** and **154**). This is the reset whose absence is the bug described in [1]; the bug-study model removes it. The UPPAAL model merges each clean-up into one update annotated with lines 116 and 150.

```
109      DEBUG_OUT("Waiting until data is processed.");
110      wait(b_transport_finished_ev);
111
112      cout << HEADER << "release_transaction" << endl;
113
114      //release transaction
115      delete current_trans;
116      current_trans = NULL;
117
118      //next request can be started.
119      current_DataLength = 0;
120      m_txn_count = 0;
121
122      //stg.write(ptr, (uint32_t)addr, 8*len);
123      tran.response_status = tlm::TLM_OK_RESPONSE;
124      break;
 ...
143      cout << HEADER << "Waiting until data is processed." << endl;
144      wait(b_transport_finished_ev);
145
146      cout << HEADER << "release_transaction" << endl;
147
148      //release transaction
149      delete current_trans;
150      current_trans = NULL;
151
152      //next request can be started.
153      current_DataLength = 0;
154      m_txn_count = 0;
155
156      tran.response_status = tlm::TLM_OK_RESPONSE;
```

c) Master adapter: bus grant received (tlm2_master_to_ahb.cpp)

The two places where the adapter records a bus grant. Our timing clock `x` is reset on exactly these two edges of the UPPAAL model.

```
 32        case tlm::TLM_UPDATED:
 33        {
 34          if(ph == BUS_GRANT)
 35          {
 36            m_bus_granted = true;
 37            m_bus_req_pending = false;
 38            cout << HEADER << "phase: BUS_GRANT, tlm_sync_enum:TLM_UPDATED" << endl;
 39          }
 ...
274    else if (phase == BUS_GRANT)
275    {
276      cout << HEADER << "nb_bw, ph:BUS_GRANT" << endl;
277      m_bus_granted = true;
278      m_bus_req_pending = false;
279
280      requestProcess();
281    }
```

d) Traffic generator: tlm2_simple_master::thread_process (tlm2_simple_master.cpp)

Each round writes and then reads a burst of BURST_LENGTH words through blocking transport and waits 20 clock edges. `defines.h` sets `MEM_SIZE` to 3 and `BURST_LENGTH` to `MEM_SIZE` (lines 4 and 15). Its `SLAVE_N` (2 AHB slaves, lines 5-7) and `MASTER_N` (2) describe the original 2M2S design; when the model was reduced to 1M1S, `number_of_slaves` in this generator was set to 1, so every burst goes to address 0 of the single slave.

```
 20    while(true)
 21    {
 22      for (unsigned int i = 0; i < slave_addr_range * number_of_slaves; i = i + 3)
 23      {
 24        for (int l = 0; l < 2; l++)
 25        {
 26
 27          burst_len = BURST_LENGTH;
 28          unsigned int *dptr = &dataArr[0];
 29
 30          // generate random address. address range must fit to slaves.
 31          //adr = rand() % (number_of_slaves*slave_addr_range);
 32
 33          adr = i % (number_of_slaves * slave_addr_range);
 ...
 84          DEBUG_OUT("Sending data ...");
 85          init_sock->b_transport(d, delay);
 86          DEBUG_OUT("Transmission finished.");
 ...
103          int wait_clks = 20;
104
105          cout << HEADER << " wait " << wait_clks << " clk's until next transport..." <<
endl;
106          for (int n = 0; n < wait_clks; n++)
107          {
108            wait(pos_clk_edge);
109          }
```

e) Code added for communication in time (tools/make_models.py)

```
// global declarations
clock x;
bool  M1_isCommunicating = false;


// on both bus-grant edges (tlm2_master_to_ahb.cpp lines 36-37 and 277-278)
m_bus_granted = true, m_bus_req_pending = false,  x = 0, M1_isCommunicating = true


// on both clean-up edges (lines 120 and 154)
... m_txn_count = 0,  M1_isCommunicating = false
```

f) Verification queries (queries/1M1S_QUERIES.q, 1M1S_TIMED_QUERIES.q, BUG_STUDY_NORESET.q)

```
A[] not deadlock
AHB_M1$m_bus_req_pending --> AHB_M1$m_bus_granted
E<> AHB_M1$m_bus_granted
A[] AHB_Bus$m_InService_Register != 2
A[] AHB_Bus$m_InService_Register <= 1
A[] not MEM_ERR_ACCESS_FAIL
A[] not MEM_ERR_DOUBLE_FREE
A[] not MEM_ERR_NOT_ENOUGH_MEM
A[] M1_isCommunicating imply x <= 40            // timed model
M1_isCommunicating --> !M1_isCommunicating        // timed and no-reset models
```

Simulation Result (Proper screenshots):

All screenshots were taken in the UPPAAL 5.0.0 GUI. The GUI uses demo copies of the models in which the source-line comment boxes are removed and "$" in names is written "__"; these copies give the same results as the models used for the numbers in the Synthesis Report.

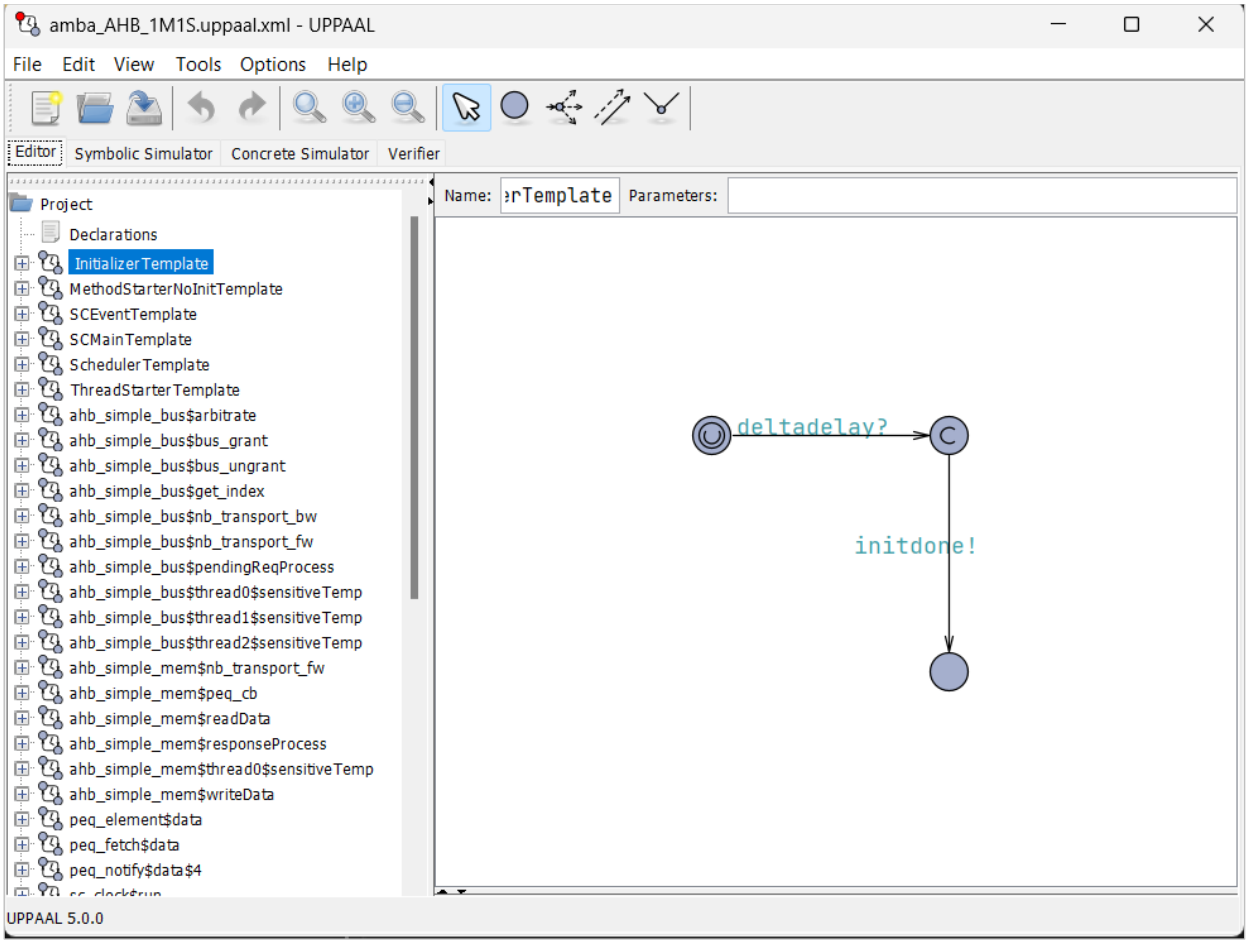


Fig. 2. The 1M1S model in the UPPAAL editor with its template list.

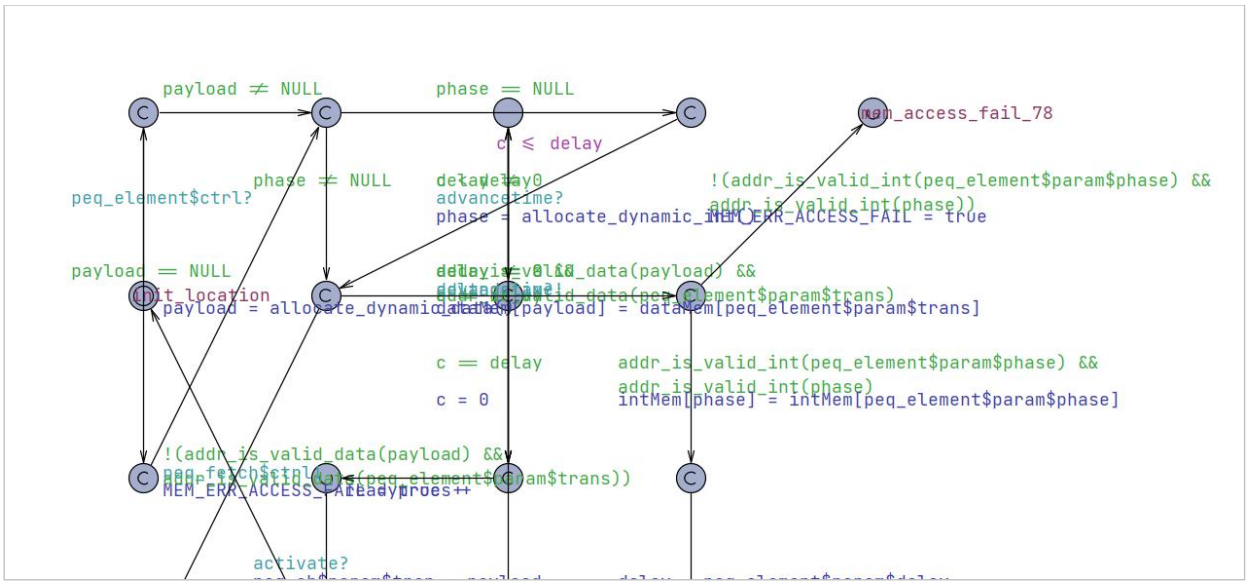


Fig. 3. `peq_element`: PEQ notification automaton with local clock c and invariant c <= delay (cf. [1, Fig. 4]).

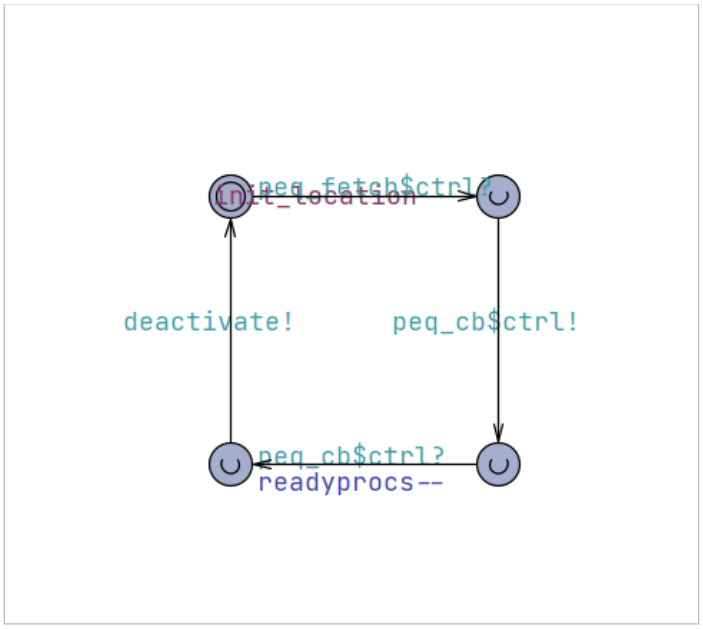


Fig. 4. `peq_fetch`: callback invocation automaton (cf. [1, Fig. 5]).

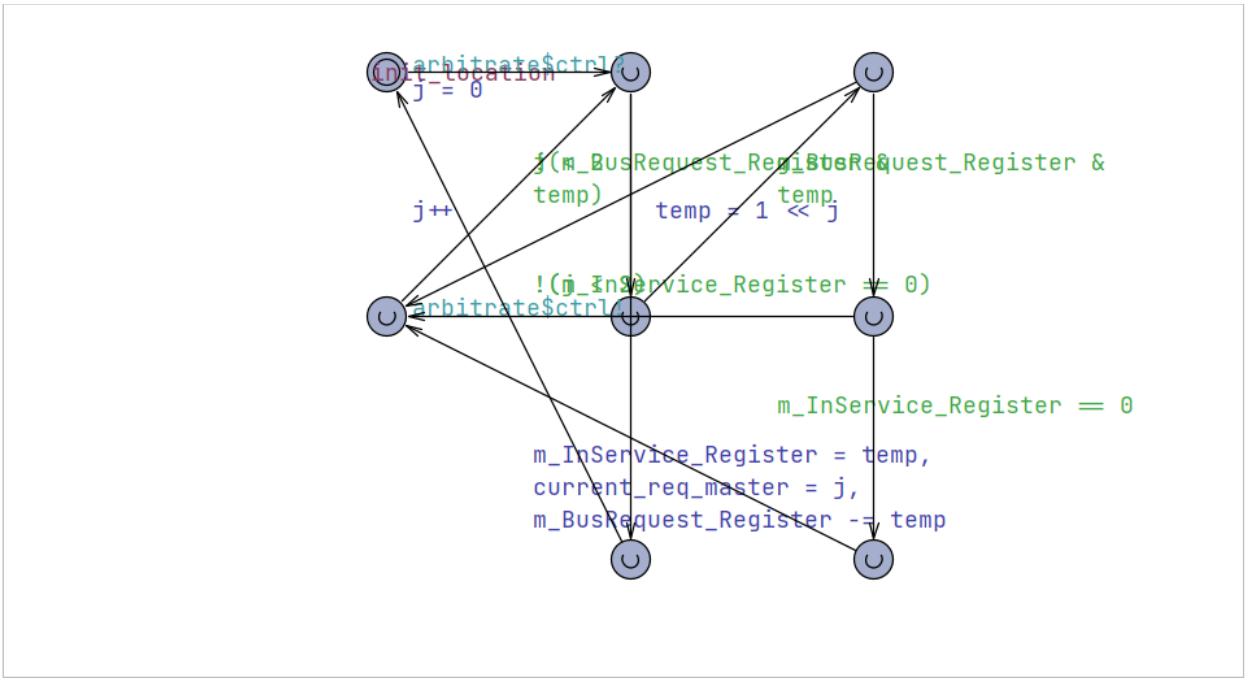


Fig. 5. `ahb_simple_bus__arbitrate`: the bus is granted only when m_InService_Register == 0.

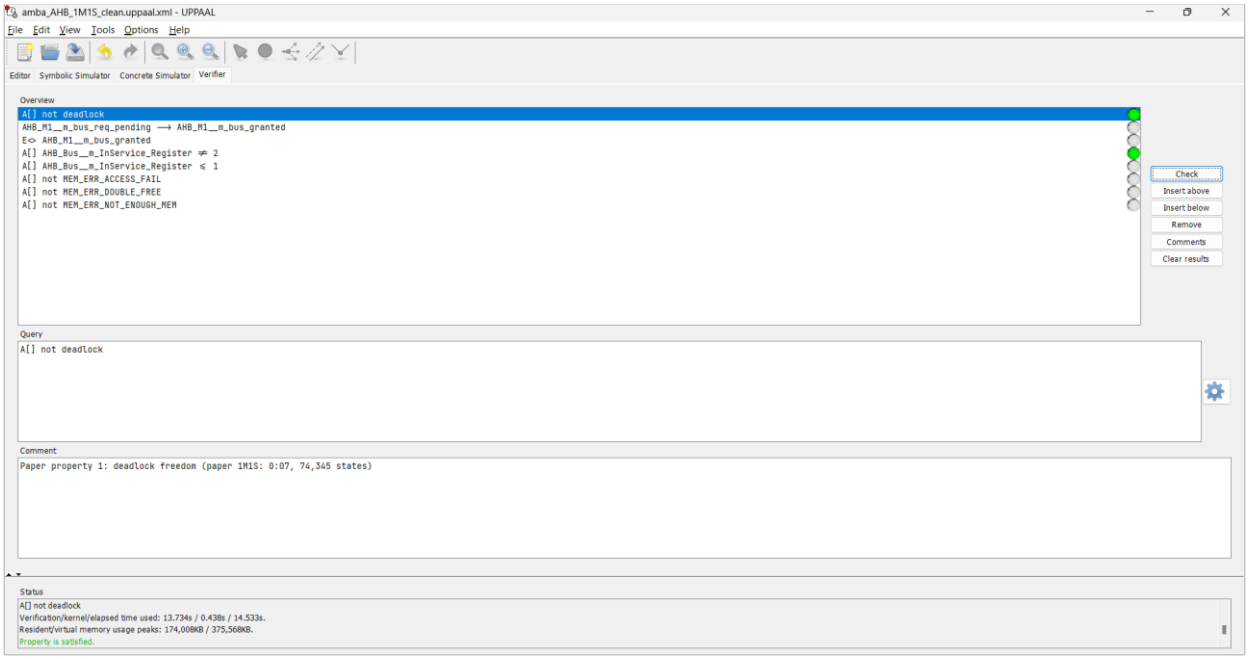


Fig. 6. Deadlock freedom: `A[] not deadlock` is satisfied.

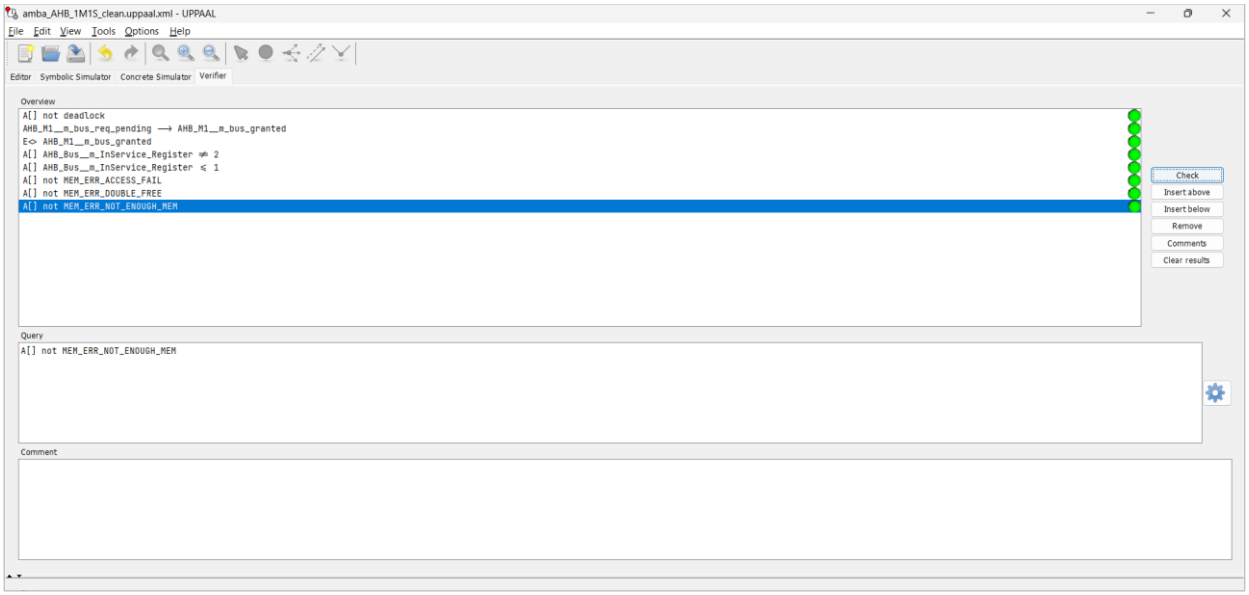


Fig. 7. All eight 1M1S queries satisfied, including the leads-to bus-grant property.

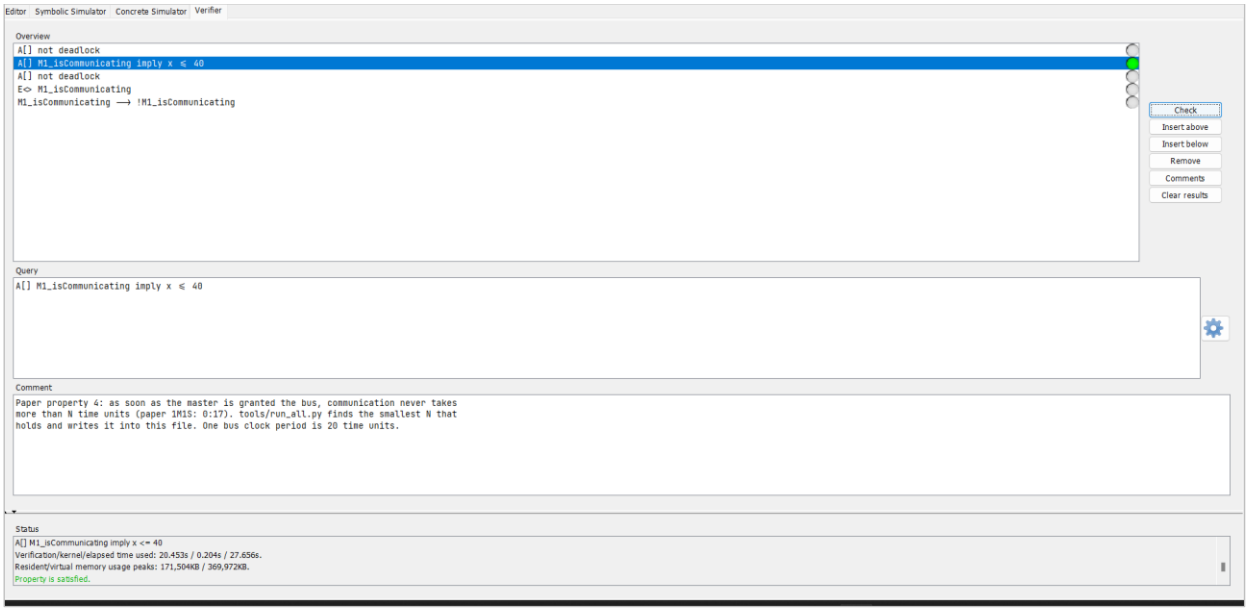


Fig. 8. Communication in time: `A[] M1_isCommunicating imply x <= 40` is satisfied.

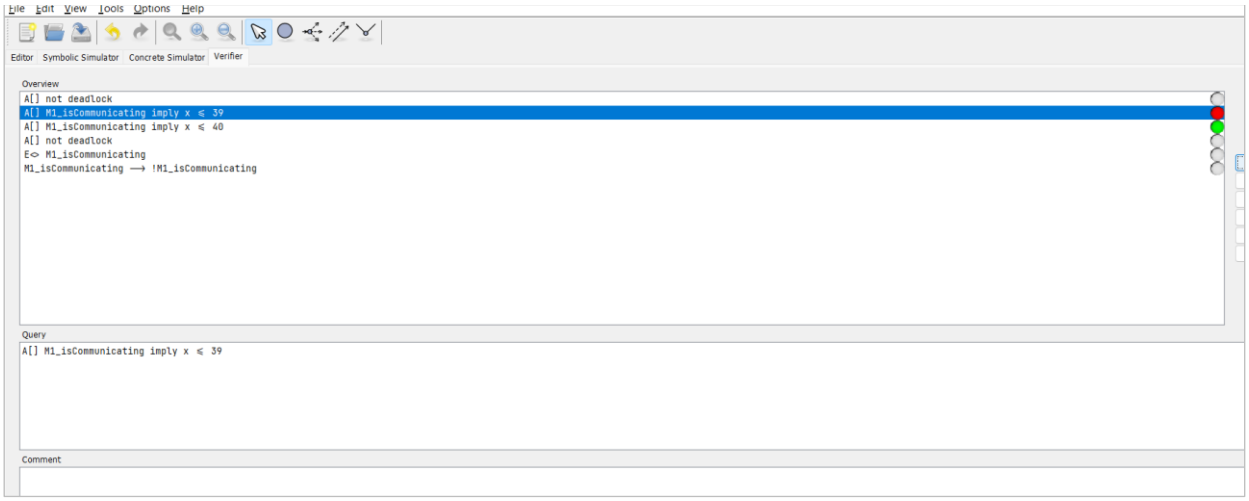


Fig. 9. `x <= 39` is not satisfied while `x <= 40` is, so N = 40 is the smallest bound.

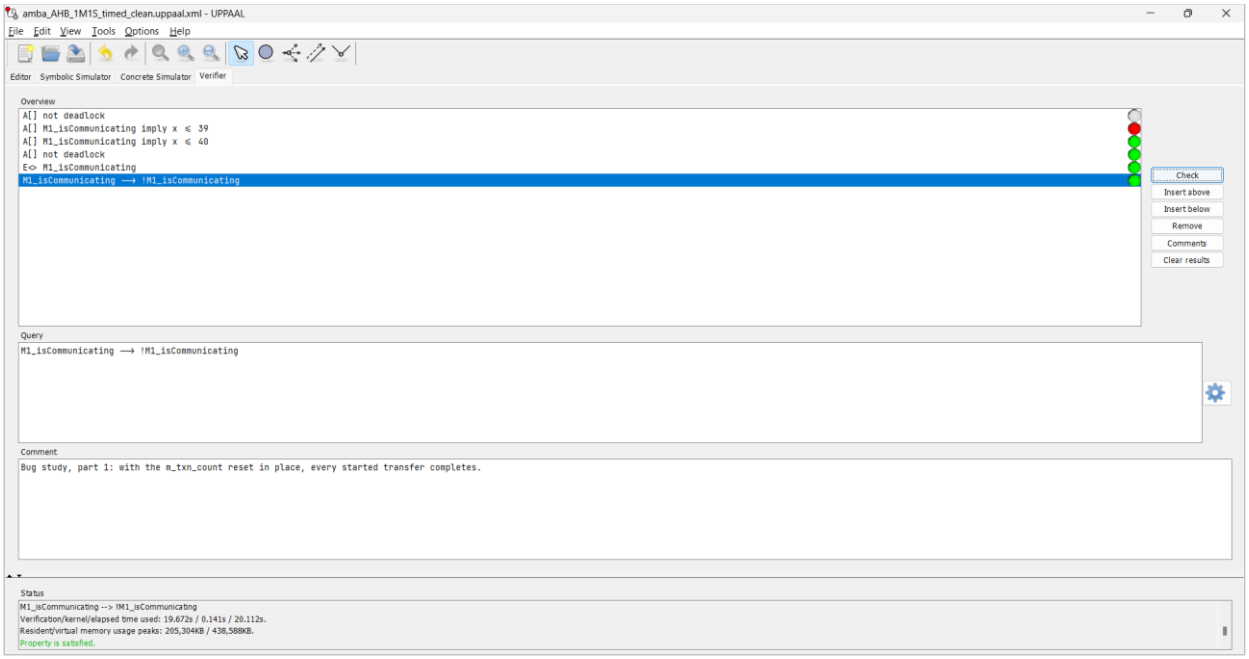


Fig. 10. Timed model: no deadlock, communication starts, and with the reset every started transfer completes.

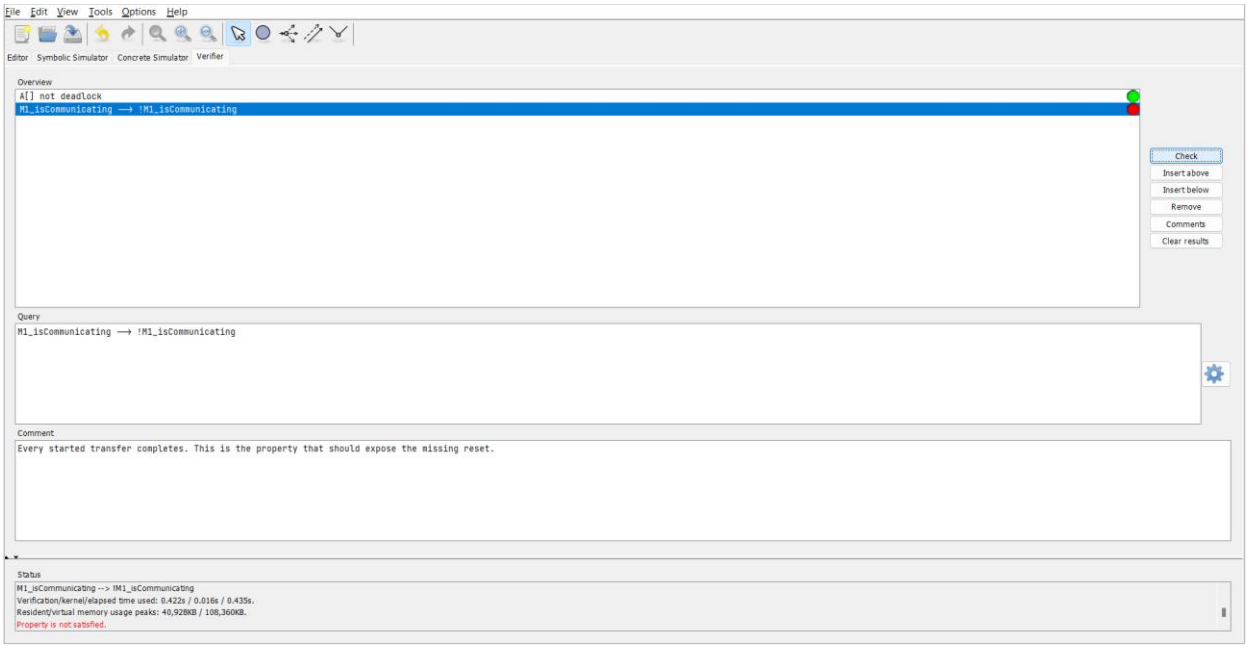


Fig. 11. Bug study: without the reset, deadlock freedom still holds but `M1_isCommunicating --> !M1_isCommunicating` is not satisfied.

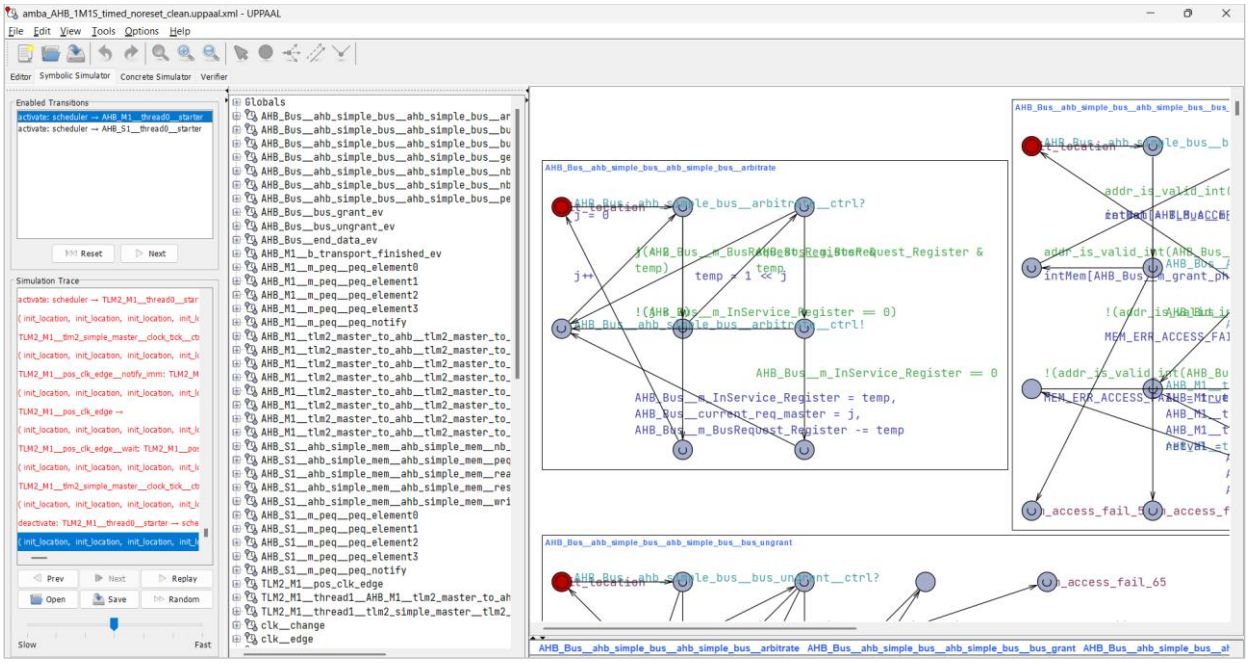


Fig. 12. The counterexample loaded into the Symbolic Simulator.

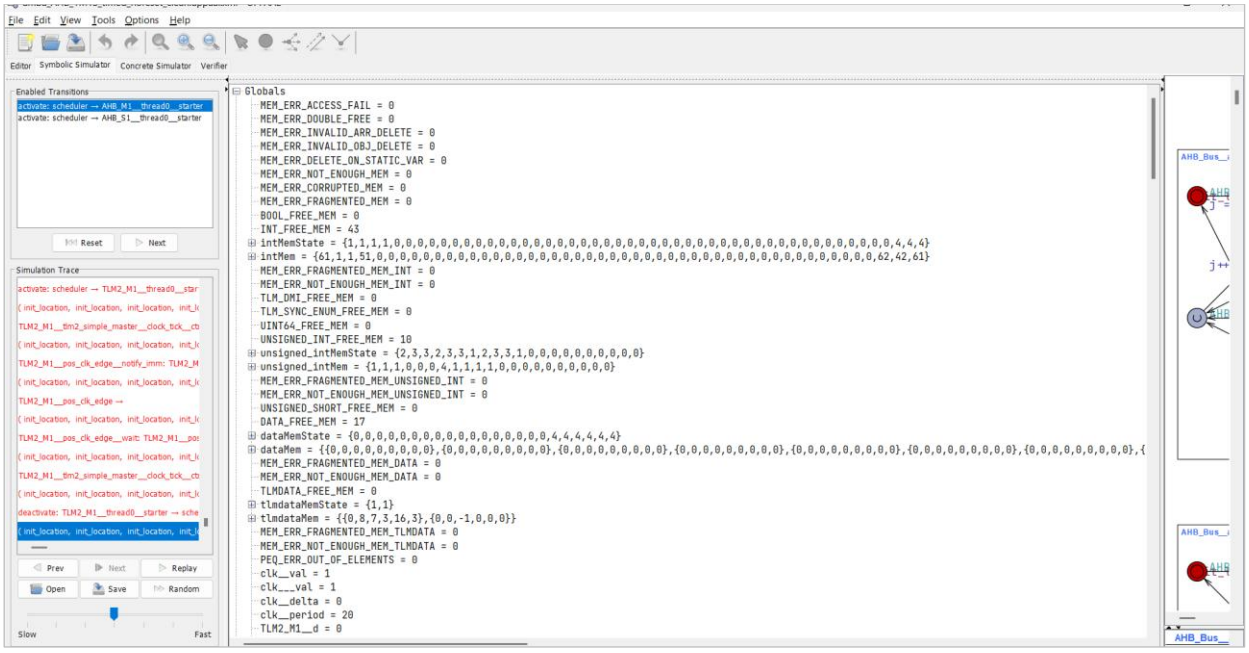


Fig. 13. Global variables at the end of the counterexample: no memory errors, clock period 20.

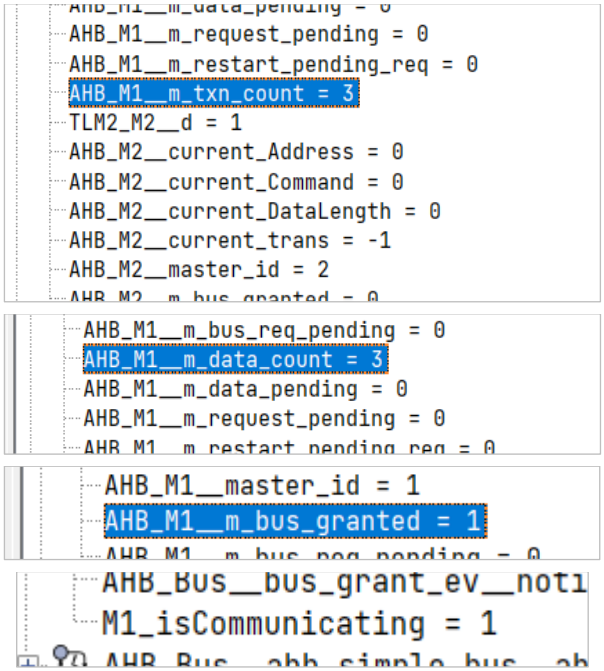


Fig. 14. Key counterexample values: `m_data_count = 3` and `m_txn_count = 3` (never reset), `m_bus_granted = 1` and `M1_isCommunicating = 1` for ever, so the transfer never finishes.

Synthesis Report (Screenshots):

This is a formal-verification assignment, so there is no logic synthesis; the equivalent report is the verification report below. All values come from `verifyta` (UPPAAL 5.0.0, run 2026-11-10 14:53) and are saved in `results/results.md` and `results/raw/`. Times are CPU time.

Table I. Paper (Table II, 1M1S) compared with our results

Property	Paper time	Paper states	Our CPU time	Our states explored	Our result
Deadlock freedom	0:07	74,345	16.38 s	1,241,387	Satisfied
Only one master granted	-	-	N/A	N/A	Not applicable (1 master)
Bus granted to M1	0:05	74,345	11.83 s	1,259,923	Satisfied
Bus granted to M2	-	-	N/A	N/A	Not applicable (1 master)
Communication in time (N = 40)	0:17	74,345	13.02 s	1,241,387	Satisfied

Our deadlock check explored 1,241,387 states (1,119,525 stored), about 16.7 times the paper's 74,345. We report both values instead of copying the paper's. Possible reasons, none of which we isolated, are the different UPPAAL version (the paper used a 2011 release), our model being reduced by hand rather than regenerated, and a PEQ depth of four entries. The paper ran on a 3.4 GHz Pentium with 4 GB RAM, so times are not comparable either.

For communication in time, `x <= 40` holds and `x <= 39` fails, so N = 40 time units (2 clock periods of 20) is the smallest bound. Because x restarts at every grant, N bounds the time from the latest grant to the end of the transaction.

Table II. Additional checks on the 1M1S and timed models

Check	Query	States explored	CPU time	Result
M1 can be granted (reachability)	`E<> AHB_M1$m_bus_granted`	501	0.01 s	Satisfied
Master-2 slot never granted	`A[] AHB_Bus$m_InService_Register != 2`	1,241,387	12.78 s	Satisfied
Grant register only 0 or 1	`A[] AHB_Bus$m_InService_Register <= 1`	1,241,387	17.16 s	Satisfied
No invalid memory access	`A[] not MEM_ERR_ACCESS_FAIL`	1,241,387	16.55 s	Satisfied
No double free	`A[] not MEM_ERR_DOUBLE_FREE`	1,241,387	36.83 s	Satisfied
No out-of-memory	`A[] not MEM_ERR_NOT_ENOUGH_MEM`	1,241,387	42.38 s	Satisfied
Timed model: deadlock freedom	`A[] not deadlock`	1,241,387	1 min 5 s	Satisfied
Timed model: communication starts	`E<> M1_isCommunicating`	501	0.02 s	Satisfied

Table III. Bug study: effect of removing the m_txn_count reset

Check	Query	States explored	CPU time	Result
With reset: transfers complete	`M1_isCommunicating --> !M1_isCommunicating`	1,799,151	1 min 13 s	Satisfied
Without reset: deadlock freedom	`A[] not deadlock`	57,806	0.69 s	Satisfied
Without reset: transfers complete	`M1_isCommunicating --> !M1_isCommunicating`	41,403	0.42 s	Not satisfied

Without the reset the model still has no deadlock, but a started transfer never finishes. The counterexample ends with `m_txn_count = 3` and `M1_isCommunicating = 1` for ever. The missing reset therefore shows up as a liveness failure, most likely because the clock and scheduler keep running. Our model has no AMBA SPLIT response, so this shows the effect of the missing reset on the generated bursts rather than the paper's exact split-transaction scenario.

Table IV. Search for the communication-in-time bound N

N (time units)	Result	CPU time
20	Not satisfied	0.03 s
30	Not satisfied	0.06 s
35	Not satisfied	0.08 s
37	Not satisfied	0.11 s
38	Not satisfied	0.08 s
39	Not satisfied	0.19 s
40	Satisfied	13.02 s

Table V. Toolchain check on the SystemC FIFO example

Check	Query	States explored	CPU time	Result
FIFO: deadlock freedom	`A[] not deadlock`	11,163	0.05 s	Satisfied
FIFO: no invalid memory access	`A[] not MEM_ERR_ACCESS_FAIL`	11,163	0.02 s	Satisfied
FIFO: no memory corruption	`A[] not MEM_ERR_CORRUPTED_MEM`	11,163	0.02 s	Satisfied

Conclusion:

We repeated the 1M1S part of the AMBA AHB experiment of [1] with UPPAAL. All three applicable properties hold: deadlock freedom (1,241,387 states explored), bus grant to master 1 as a leads-to property, and communication in time with the smallest bound N = 40 time units (2 clock periods). The arbiter never grants the unused master-2 slot and the model has no memory errors. Our state count is about 16.7 times the paper's, which we report openly. The bug study shows that removing the `m_txn_count` reset does not cause a deadlock but makes transfers hang, which only a liveness property detects. Every number in this report is produced by a script from saved UPPAAL output, so the work can be reproduced with one command.

Limitations: the 1M1S model was reduced by hand from the 2M2S model rather than regenerated from the SystemC source, and the communication-in-time bound is measured from the latest grant (measuring from another start point, such as the start of b_transport, gives a different bound). Extending the work to 2M2S would need far more memory, which is why this assignment is limited to 1M1S.

Reference:

[1] M. Pockrandt, P. Herber, and S. Glesner, "Model checking a SystemC/TLM design of the AMBA AHB protocol," in Proc. 9th IEEE Symp. Embedded Systems for Real-Time Multimedia (ESTIMedia), Taipei, Taiwan, 2011.

[2] P. Herber, M. Pockrandt, and S. Glesner, "Transforming SystemC transaction level models into UPPAAL timed automata," in Proc. 9th ACM/IEEE Int. Conf. Formal Methods and Models for Codesign (MEMOCODE), 2011, pp. 161-170.

[3] P. Herber, J. Fellmuth, and S. Glesner, "Model checking SystemC designs using timed automata," in Proc. Int. Conf. Hardware/Software Codesign and System Synthesis (CODES+ISSS), 2008, pp. 131-136.

[4] G. Behrmann, A. David, and K. G. Larsen, "A tutorial on UPPAAL," in Formal Methods for the Design of Real-Time Systems, LNCS 3185, Springer, 2004, pp. 200-236.

[5] ARM Ltd., AMBA Specification (Rev 2.0), ARM IHI 0011A, 1999.

[6] IEEE Standard for Standard SystemC Language Reference Manual, IEEE Std 1666-2011.

[7] R. Alur and D. L. Dill, "A theory of timed automata," Theoretical Computer Science, vol. 126, pp. 183-235, 1994.